STAGE 1

In this stage I build a tokenizer from scratch for data preparation and sampling purposes.

Here for dataset I am using WikiText-2 from huggingface and TinyStories.

Steps followed usually :

1. Split the text into individual word and subwords
2. Convert these tokens into token IDs.

```
! pip install datasets
```

```
Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (4.0.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets) (3.25.2)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.0.2)
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (18.1.0)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.32.4)
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.12/dist-packages (from datasets) (4.67.3)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.6.0)
Requirement already satisfied: multiprocess<0.70.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3
Requirement already satisfied: huggingface-hub>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (1.10.1)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from datasets) (26.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from datasets) (6.0.3)
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.
Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->da
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->datase
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->datasets) (0.24.1
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datas
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2.9.0.
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2026.1)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fss
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fs
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fss
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fs
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.24.0-
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->huggi
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->dataset
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.24.0->dat
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.24.
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.24.0->dat
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.2
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->huggi
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->hug
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3.0->
```

```
from datasets import load_dataset

wiki = load_dataset("wikitext", "wikitext-2-raw-v1")
print(wiki)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
README.md:        10.5k/?  [00:00<00:00, 167kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
wikitext-2-raw-v1/test-00000-of-00001.pa(…): 100%  733k/733k  [00:00<00:00, 3.66MB/s]
wikitext-2-raw-v1/train-00000-of-00001.p(…): 100%  6.36M/6.36M  [00:00<00:00, 31.7MB/s]
wikitext-2-raw-v1/validation-00000-of-00(…): 100%  657k/657k  [00:00<00:00, 2.78MB/s]
Generating test split: 100%  4358/4358  [00:00<00:00, 60354.28 examples/s]
Generating train split: 100%  36718/36718  [00:00<00:00, 365503.72 examples/s]
Generating validation split: 100%  3760/3760  [00:00<00:00, 120634.77 examples/s]
DatasetDict({
    test: Dataset({
        features: ['text'],
        num_rows: 4358
    })
    train: Dataset({
        features: ['text'],
        num_rows: 36718
    })
    validation: Dataset({
        features: ['text'],
        num_rows: 3760
    })
})
```

We can see here its a dataset dict. For encoder we only need the train part.

```
wiki_text = wiki['train']['text']
print(repr(wiki_text[0]))
```

```
''
```

```
for i in range(5):
  print(repr(wiki_text[i]))
```

```
''
' = Valkyria Chronicles III = \n'
''
' Senjō no Valkyria 3 : Unrecorded Chronicles ( Japanese : 戦場のヴァルキュリア3 , lit . Valkyria of the Battlefield 3 ) , commonl
" The game began development in 2010 , carrying over a large portion of the work done on Valkyria Chronicles II . While it retai
```

Alright first i thought something was wrong but it seems everything is working correctly.

Looks good to me now i think i can move forward.

```
for i in range(10):
  print(repr(wiki_text[i]))
```

```
''
' = Valkyria Chronicles III = \n'
''
' Senjō no Valkyria 3 : Unrecorded Chronicles ( Japanese : 戦場のヴァルキュリア3 , lit . Valkyria of the Battlefield 3 ) , commonl
" The game began development in 2010 , carrying over a large portion of the work done on Valkyria Chronicles II . While it retai
" It met with positive sales in Japan , and was praised by both Japanese and western critics . After release , it received downl
''
' = = Gameplay = = \n'
''
" As with previous Valkyira Chronicles games , Valkyria Chronicles III is a tactical role @-@ playing game where players take co
```

It seems as if theres a lot of empty spaces i should clean them first.

```
wiki_clean = [item for item in wiki_text if item.strip() != ' ']
for i in range(10):
```

```
print(wiki_clean[i])
```

```
= Valkyria Chronicles III =

Senjō no Valkyria 3 : Unrecorded Chronicles ( Japanese : 戦場のヴァルキュリア3 , lit . Valkyria of the Battlefield 3 ) , commonly

The game began development in 2010 , carrying over a large portion of the work done on Valkyria Chronicles II . While it retain

It met with positive sales in Japan , and was praised by both Japanese and western critics . After release , it received downlo

= = Gameplay = =

As with previous Valkyira Chronicles games , Valkyria Chronicles III is a tactical role @-@ playing game where players take con
```

Ideas

I am keeping the *= = = Titles = = =* sections in hopes my model will learn these mean headings or titles and might print

= = = Knight's Voyage = = =

or something like that

```
ds = load_dataset("roneneldan/TinyStories")
```

```
README.md:          1.06k/?  [00:00<00:00,  70.6kB/s]
data/train-00000-of-00004-2d5a1467fff108(…):  100%  249M/249M  [00:02<00:00,  221MB/s]
data/train-00001-of-00004-5852b56a2bd28f(…):  100%  248M/248M  [00:01<00:00,  242MB/s]
data/train-00002-of-00004-a26307300439e9(…):  100%  246M/246M  [00:03<00:00,  185MB/s]
data/train-00003-of-00004-d243063613e5a0(…):  100%  248M/248M  [00:02<00:00,  249MB/s]
data/validation-00000-of-00001-869c898b5(…):  100%  9.99M/9.99M  [00:00<00:00,  50.0MB/s]
Generating  train  split:  100%  2119719/2119719  [00:12<00:00,  191620.65  examples/s]
Generating  validation  split:  100%  21990/21990  [00:00<00:00,  182749.83  examples/s]
```

```
print(type(ds))
```

```
<class 'datasets.dataset_dict.DatasetDict'>
```

```
stories_text = list(ds['train']['text'])
print(type(stories_text))
print(stories_text[10])
```

```
<class 'list'>
Once upon a time, there was a big car named Dependable. He had a very important job. Dependable would take a family to the park

One day, when they got to the park, they saw a big sign that said, "Fun Race Today!" The family was very excited. They knew that

The race started, and Dependable went very fast. The other cars tried to catch up, but Dependable was too quick. In the end, Dep
```

```
dataset = wiki_clean + stories_text # I forgot to convert both types hehe :p
```

```
print(len(dataset))
```

```
2156437
```

Sweet

Now we build the tokenizer.

But before that I will make a .py file.

PS: Now I think i should remove " *==== Titles ===* "

I can include them in finetuning

```
dataset = [item for item in dataset if not item.strip().startswith("=")]
```

```
#A pretty stupid line
print("Searching if dataset has lines starting with '= = = Title = = =' ")
for i in range(len(dataset)):
  if dataset[i].strip().startswith("="):
    print(dataset[i])

print(". . .\n")

print("Searching complete lets look at some samples")
for i in range(5):
  print(dataset[i])
```

```
Searching if dataset has lines starting with '= = = Title = = ='
. . .

Searching complete lets look at some samples


 Senjō no Valkyria 3 : Unrecorded Chronicles ( Japanese : 戦場のヴァルキュリア3 , lit . Valkyria of the Battlefield 3 ) , commonly

 The game began development in 2010 , carrying over a large portion of the work done on Valkyria Chronicles II . While it retain

 It met with positive sales in Japan , and was praised by both Japanese and western critics . After release , it received downlo
```

LOL it didnt print anything but contents did print stuff

For the time being i save my dataset

```
"""final_dataset = " ".join(dataset)
print(final_dataset[1:9])"""
```

```
'final_dataset = " ".join(dataset)\nprint(final_dataset[1:9])'
```

```
"""print(final_dataset[:100])"""
```

```
'print(final_dataset[:100])'
```

tiktoken is a library that helps us use BPE. We instantiate BPE.

```
# ! pip install tiktoken
```

```
"""import importlib
import tiktoken"""
```

```
'import importlib\nimport tiktoken'
```

```
"""tokenizer = tiktoken.get_encoding("gpt2")"""
```

```
'tokenizer = tiktoken.get_encoding("gpt2")'
```

Every tokenizer has encode method and decode method.

This is good enough but I'd like to make my own tokenizer.

```
print(len(dataset))
```

```
2150225
```

```
vocab = []

for line in range(499999):
    words = dataset[line].split()
    for word in words:
        vocab.append(" ".join(list(word)) + " </w>")
```

```
print(len(vocab))
```

```
84653495
```

```
print(vocab[10])
```

```
戦 場 の ヴ ァ ル キ ュ リ ア 3 </w>
```

What the-

```
print(vocab[1010])
```

```
c h a r a c t e r s </w>
```

```
example = len(set(vocab))
print(example)
```

```
205773
```

```
from collections import Counter
```

```
print(type(vocab))
```

```
<class 'list'>
```

Cant work with this need to make changes

```
vocab_freq = Counter(vocab)
print(len(vocab_freq))
print(type(vocab_freq))
```

```
205773
<class 'collections.Counter'>
```

```
for i, (word, freq) in enumerate(vocab_freq.items()):
    print(word, ":", freq)
    if i == 10:
        break
```

```
S e n j ō </w> : 5
n o </w> : 49296
V a l k y r i a </w> : 53
3 </w> : 10561
: </w> : 3744
U n r e c o r d e d </w> : 1
C h r o n i c l e s </w> : 46
( </w> : 11744
J a p a n e s e </w> : 256
戦 場 の ヴ ァ ル キ ュ リ ア 3 </w> : 3
, </w> : 99802
```

```
"""import re, collections
def get_stats(vocab):
  pairs = collections.defaultdict(int)
  for word, freq in vocab.items():
    symbols = word.split()
  for i in range(len(symbols)-1):
    pairs[symbols[i],symbols[i+1]] += freq
  return pairs

def merge_vocab(pair, v_in):
  v_out = {}
  bigram = re.escape(' '.join(pair))
  p = re.compile(r'(?<!\S)' + bigram + r'(?!\S)')
  for word in v_in:
    w_out = p.sub(''.join(pair), word)
    v_out[w_out] = v_in[word]
  return v_out
vocab = {'l o w </w>' : 5, 'l o w e r </w>' : 2,
'n e w e s t </w>':6, 'w i d e s t </w>':3}


num_merges = 10
```

```
for i in range(num_merges):
  pairs = get_stats(vocab)
  print(pairs)
  best = max(pairs, key=pairs.get())
  vocab = merge_vocab(best, vocab)
  print(best)

# print(vocab)"""
```

```
<>:13: SyntaxWarning: invalid escape sequence '\S'
<>:13: SyntaxWarning: invalid escape sequence '\S'
/tmp/ipykernel_1498/4246427281.py:13: SyntaxWarning: invalid escape sequence '\S'
  p = re.compile(r'(?<!\S)' + bigram + r'(?!\S)')
'import re, collections\ndef get_stats(vocab):\n  pairs = collections.defaultdict(int)\n  for word, freq in vocab.items():\n
symbols = word.split()\n  for i in range(len(symbols)-1):\n    pairs[symbols[i],symbols[i+1]] += freq\n  return pairs\n\ndef me
rge_vocab(pair, v_in):\n  v_out = {}\n  bigram = re.escape(' '.join(pair))\n  p = re.compile(r'(?<!\\S)' + bigram + r'(?!\\S)')
\n  for word in v_in:\n    w_out = p.sub(''.join(pair), word)\n    v_out[w_out] = v_in[word]\n  return v_out\nvocab = {'l o w
```

```
for word in range(1):
  subword = vocab[word].split()
  print(vocab[word])
  print(subword)
```

```
S e n j ō </w>
['S', 'e', 'n', 'j', 'ō', '</w>']
```

```
def get_pair(vocab_freq):

  pairs = {}

  for word,freq in vocab_freq.items():
    subword = word.split()
    for i in range(len(subword)-1):
      pair = (subword[i],subword[i+1])
      pairs[pair] = pairs.get(pair, 0) + freq

  return pairs
```

```
pairs = get_pair(vocab_freq)
```

```
for i, (subword, freq) in enumerate(pairs.items()):
    print(subword, ":", freq)
    if i == 10:
        break
```

```
('S', 'e') : 9614
('e', 'n') : 2619496
('n', 'j') : 39458
('j', 'ō') : 6
('ō', '</w>') : 34
('n', 'o') : 930499
('o', '</w>') : 4302400
('V', 'a') : 1251
('a', 'l') : 1477810
('l', 'k') : 177376
('k', 'y') : 98173
```

```
print(len(pairs))
```

```
6752
```

```
print(type(pairs))
```

```
<class 'dict'>
```

```
best = max(pairs, key=pairs.get)
print(best)
print("Best pair : {" ,best , pairs[best],"}")
```

```
('e', '</w>')
Best pair : { ('e', '</w>') 15410043 }
```

```
for i, (word,freq) in enumerate(vocab_freq.items()):
  print(word,type(word), ":", freq)
```

```
  if i == 10:
    break
```

```
def merge_vocab(word_pair, vocab_in):
      keysub = " ".join(word_pair)
      replacement = "".join(word_pair)

      new_vocab = {}

      for word, freq in vocab_in.items():
            item = 0
            new_token = []
            temp_char_stream = word.split()

            while item < len(temp_char_stream) - 1:
                comparison = temp_char_stream[item] + " " + temp_char_stream[item + 1]

                if comparison == keysub:
                    new_token.append(replacement)
                    item += 2
                else:
                    new_token.append(temp_char_stream[item])
                    item += 1

            final_word = " ".join(new_token)
            new_vocab[final_word] = freq

      return new_vocab
```

```
new_freq = merge_vocab(best, vocab_freq)
```

```
for i,(key,freq) in enumerate(new_freq.items()):
  if "".join(best) in key:
    print(key, ":", freq)
  if i==100:
    break
```

```
print(len(new_freq))
```

```
class Trainer():
  def __init__(self, vocab_freq, num_merges):
    self.vocab_freq = vocab_freq
    self.num_merges = num_merges
    # Thinking about adding self.merges = [] to persist learned rules for encode()function


  def get_best_pair(self,vocab_in):
    pairs = {}
    for word,freq in vocab_in.items():
      subword = word.split()
      for i in range(len(subword)-1):
        pair = (subword[i],subword[i+1])
        pairs[pair] = pairs.get(pair, 0) + freq
    # Edge case: if pairs is empty, max() function will crash -> handle or early-return in loop()
    # Thinking about adding if len of .split() == 1 then we break
    best_pair = max(pairs, key=pairs.get)
    return best_pair


  def merge_vocab(self, word_pair, vocab_input):
      keysub = " ".join(word_pair)
      replacement = "".join(word_pair)
      new_vocab = {}
      for word, freq in vocab_input.items():
            item = 0
            new_token = []
            temp_char_stream = word.split()
            while item < len(temp_char_stream) - 1 :
                comparison = temp_char_stream[item] + " " + temp_char_stream[item + 1]
                if comparison == keysub:
                    new_token.append(replacement)
                    item += 2
                else:
                    new_token.append(temp_char_stream[item])
```

```
                        item += 1
                # Bug: last token can be dropped -> ensure trailing
                # token is appended when loop ends
                # Kinda confused
                final_word = " ".join(new_token)
                new_vocab[final_word] = freq
            return new_vocab


      def loop(self):
          best_pairs = []
          iters = self.num_merges
          for i in range(iters):
            vocab_input = self.vocab_freq
            best_at_this_iter = self.get_best_pair(vocab_input)
            best_pairs.append(best_at_this_iter)
            # Optional feature(from before): also append to self.merges if possibel
            # for later encode() function
            new_freq = self.merge_vocab(best_at_this_iter, vocab_input)
            self.vocab_freq = new_freq
            # Edge case: if no valid pairs remain, considering breaking early
            # That loop problem with max() function
          # Design: returning merges is good; also considering storing
          # them in the class
          return best_pairs

"""So many things to change this is so fascinating"""
```

Start coding or generate with AI.